Project Documentation and Workarounds

This document provides supplementary information regarding deployment decisions and workarounds implemented to successfully complete the project steps within the constraints of the lab environment.

1. Database Setup Verification Workaround

The project requires the setup of an Aurora Serverless PostgreSQL database and verification of the vector extension and table creation.

Issue Encountered

When attempting to run the SQL commands (from scripts/aurora_sql.sql) or verify the schema using the **Amazon RDS Console Query Editor**, the following permission error was consistently encountered:

Action: dbqms:CreateQueryHistory
Context: no identity-based policy allows the action

This error indicates that the Assumed Role provided by the lab environment lacked the necessary permissions for the underlying AWS service (DBQMS - Database Query Metadata Service) used by the Query Editor feature. Since the role policies could not be modified by the user, this console method was blocked.

Resolution and Proof of Completion

To successfully complete the database setup and provide the required proof, the official method was switched from the console to the AWS Command Line Interface (CLI) using the RDS Data API.

- **Method Used:** aws rds-data execute-statement commands were executed to run the required SQL queries and perform subsequent verification.
- **Proof Provided:** The original requirement for the rds_query_editor_table_creation.png screenshot was replaced with a CLI output screenshot (cli_aurora_table_creation_proof.png) demonstrating the successful existence of the necessary database objects.
    - Verification queries confirmed the presence of the **vector** extension and the **embedding** column with the correct **vector(1536)** type in the bedrock_integration.bedrock_kb table.

2. Bedrock Application Access Fix

The Streamlit application initially failed to authenticate with the Bedrock API, resulting in a botocore.exceptions.NoCredentialsError, followed by an AccessDeniedException when trying to invoke the model.

Issue Encountered

1. **Credential Issue:** The application, when deployed, could not find the default credentials (no IAM role was attached to the execution environment).
2. **Permission Issue:** Even when running in an environment with an inherited role, that role lacked the necessary Bedrock permissions (bedrock:InvokeModel).

Resolution

1. **Authentication:** A new, dedicated IAM User was created, and its Access Key ID and Secret Access Key were secured in a **.streamlit/secrets.toml** file. The application code (bedrock_utils.py) was modified to explicitly load credentials from this file, resolving the NoCredentialsError.
2. **Authorization:** The newly created IAM User was granted the **AmazonBedrockFullAccess** AWS Managed Policy. This resolved the AccessDeniedException and enabled the Streamlit application to successfully invoke models (for validation and response generation) and query the Knowledge Base at runtime.

3. Knowledge Base Data Sync Failure

Initial attempts to sync the Knowledge Base failed with a 403 Access Denied error, specifically stating that the Knowledge Base's Service Role could not access the **amazon.titan-embed-text-v1** model.

Resolution

This issue was resolved by addressing the **Account-Level Model Access** requirement:

- Access to the **Titan Embeddings G1 - Text** model was requested and granted through the Amazon Bedrock console's Model Access page.
- Once enabled at the account level, the existing Knowledge Base Service Role automatically gained the ability to use the model, and the subsequent data sync operation succeeded, making the documents available for retrieval.